

FICHE DE PROCEDURE

AP4 - Mise en place de PrestaLab et Webvisite

Reverse proxy Caddy avec WAF Coraza

Element	Information
Contexte	Infrastructure web securisee GSB
Serveur	WEBLAB - 10.50.0.5
Acces externe	Fortigate - 172.20.104.70
Applications	PrestaLab / Webvisite
Technologies	Docker, Caddy, Coraza WAF, PrestaShop, MariaDB, PHP/Apache

Sommaire

Partie	Contenu
I	Objectif et contexte
II	Architecture generale
III	Pre-requis et arborescence
IV	Configuration de PrestaLab
V	Configuration de Webvisite
VI	Configuration de Caddy et Coraza
VII	Configuration reseau et Fortigate
VIII	Deploiement et commandes utiles
IX	Tests de validation et securite
X	Conclusion

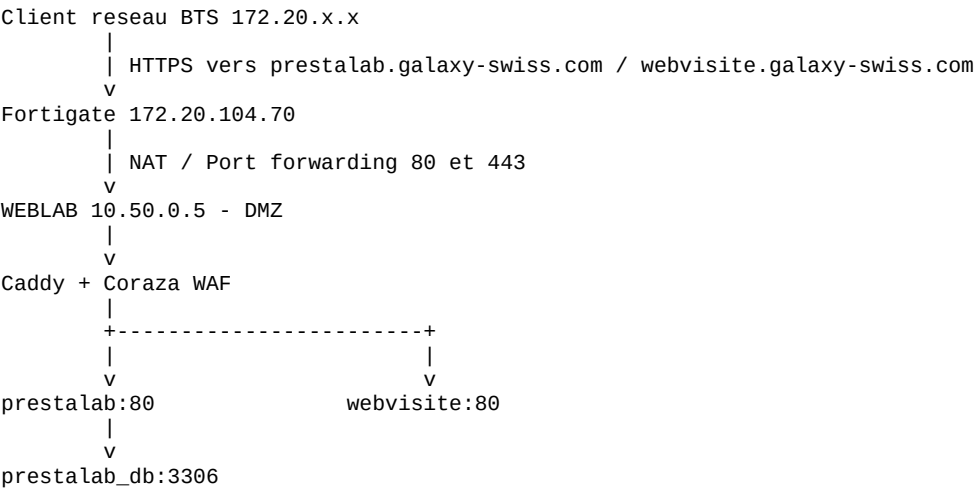
I. Objectif et contexte

L'objectif est de publier deux applications web, PrestaLab et Webvisite, depuis le serveur WEBLAB place en DMZ. Les applications sont conteneurisees avec Docker et ne doivent pas etre exposees directement au reseau client.

Les acces passent par le routeur Fortigate, puis par Caddy. Caddy joue le role de reverse proxy et integre Coraza WAF pour filtrer les requetes malveillantes simples.

- prestalab.galaxy-swiss.com -> conteneur prestalab:80
- webvisite.galaxy-swiss.com -> conteneur webvisite:80
- HTTPS active avec certificat interne Caddy
- Blocage attendu des requetes XSS et SQLi de test

II. Architecture generale



Role des composants

Composant	Role
Fortigate	Redirige les ports 80/443 du reseau BTS vers WEBLAB et filtre les flux.
WEBLAB	Serveur Debian hebergeant Docker et les conteneurs.
Caddy	Point d entree HTTP/HTTPS et reverse proxy vers les applications.
Coraza WAF	Analyse les requetes avant transmission aux applications.
Docker	Isole les services et permet la communication par noms de conteneurs.

Principe de securite : le client ne contacte jamais directement les conteneurs applicatifs. Les flux passent toujours par Fortigate puis Caddy.

III. Pre-requis et arborescence

Pre-requis

- Serveur WEBLAB operationnel dans la DMZ : 10.50.0.5.
- Docker et Docker Compose installes.
- Acces administrateur au serveur.
- Acces au Fortigate pour creer les VIP et policies.
- Fichiers applicatifs de Webvisite et configuration PrestaLab disponibles.
- Nom de domaine ou fichier hosts configure sur le poste client.

Arborescence retenue

```
/srv/web/  
├── caddy/  
│   ├── Dockerfile  
│   ├── docker-compose.yml  
│   └── Caddyfile  
├── prestashop/  
│   └── docker-compose.yml  
└── webvisite/  
    ├── Dockerfile  
    ├── docker-compose.yml  
    └── app/  
        ├── index.php  
        └── appliFrais_v2/
```

Le dossier /srv/web est utilise pour regrouper les fichiers de configuration et les sources du projet. Cela permet de separer clairement les composants Caddy, PrestaLab et Webvisite.

IV. Configuration de PrestaLab

PrestaLab est basé sur PrestaShop et utilise MariaDB. Les deux services sont placés sur le même réseau Docker afin que PrestaShop puisse joindre sa base par le nom `prestalab_db`.

Fichier `/srv/web/prestashop/docker-compose.yml`

```
services:
  prestalab_db:
    image: mariadb:10.6
    container_name: prestalab_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: <MOT_DE_PASSE>
      MYSQL_DATABASE: prestalab
      MYSQL_USER: prestalabuser
      MYSQL_PASSWORD: <MOT_DE_PASSE>
    volumes:
      - prestalab_db_data:/var/lib/mysql
    networks:
      - prestanet

  prestalab:
    image: prestashop/prestashop:latest
    container_name: prestalab
    restart: always
    depends_on:
      - prestalab_db
    environment:
      DB_SERVER: prestalab_db
      DB_NAME: prestalab
      DB_USER: prestalabuser
      DB_PASSWD: <MOT_DE_PASSE>
      PS_DOMAIN: prestalab.galaxy-swiss.com
      PS_ENABLE_SSL: 1
    volumes:
      - prestalab_data:/var/www/html
    networks:
      - prestanet

volumes:
  prestalab_db_data:
  prestalab_data:

networks:
  prestanet:
    name: prestashop-prestanet
```

Points importants

- Les mots de passe doivent être masqués dans la documentation finale.
- Le volume `prestalab_db_data` conserve les données MariaDB.
- Le domaine PrestaShop doit correspondre au nom publié dans Caddy.

V. Configuration de Webvisite

Webvisite est l'application PHP fournie. Elle est intégrée dans une image Docker basée sur PHP 8.2 avec Apache. Les extensions MySQL sont installées pour permettre les connexions à la base.

Dockerfile Webvisite

```
FROM php:8.2-apache

RUN docker-php-ext-install pdo_mysql mysqli

COPY ./app /var/www/html

RUN chown -R www-data:www-data /var/www/html \
    && find /var/www/html -type d -exec chmod 755 {} \; \
    && find /var/www/html -type f -exec chmod 644 {} \;
```

docker-compose.yml Webvisite

```
services:
  webvisite:
    build: .
    container_name: webvisite
    restart: always
    networks:
      - prestanet

networks:
  prestanet:
    external: true
    name: prestashop-prestanet
```

Justification

- Aucun port n'est exposé directement sur WEBLAB.
- Le conteneur est accessible uniquement par Caddy via le réseau Docker.
- Les permissions sont fixées pour permettre à Apache de lire correctement les fichiers.

Cette configuration évite un accès direct du client vers Webvisite et impose le passage par le reverse proxy.

VI. Configuration de Caddy et Coraza

Caddy est compile avec le module Coraza afin d agir comme reverse proxy et WAF. Il recoit les requetes HTTPS, applique les regles de securite puis transfere vers le bon conteneur.

Dockerfile Caddy avec Coraza

```
FROM caddy:builder AS builder
RUN xcaddy build --with github.com/corazawaf/coraza-caddy/v2
FROM caddy:latest
COPY --from=builder /usr/bin/caddy /usr/bin/caddy
```

docker-compose.yml Caddy

```
services:
  caddy:
    build: .
    container_name: caddy
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - prestanet

volumes:
  caddy_data:
  caddy_config:

networks:
  prestanet:
    external: true
    name: prestashop_prestanet
```

Caddyfile - principe

```
{
  order coraza_waf first
}

prestalab.galaxy-swiss.com {
  tls internal
  coraza_waf { load_owasp_crs }
  reverse_proxy prestalab:80
}

webvisite.galaxy-swiss.com {
  tls internal
  coraza_waf { load_owasp_crs }
  reverse_proxy webvisite:80
}
```

VII. Configuration reseau et Fortigate

Le Fortigate assure la redirection des flux HTTP et HTTPS depuis le reseau BTS vers WEBLAB dans la DMZ. Les VIP doivent mapper les ports 80 et 443 vers 10.50.0.5.

Element	Valeur
Reseau BTS	172.20.96.0/20
IP Fortigate cote BTS	172.20.104.70
Serveur WEBLAB DMZ	10.50.0.5
Port HTTP	172.20.104.70:80 -> 10.50.0.5:80
Port HTTPS	172.20.104.70:443 -> 10.50.0.5:443

Regles a creer

Objet	Configuration
VIP_Weblab_HTTP	TCP 80 externe vers TCP 80 interne sur 10.50.0.5
VIP_Weblab_HTTPS	TCP 443 externe vers TCP 443 interne sur 10.50.0.5
Policy Firewall	wan1 -> DMZ, destination VIP HTTP/HTTPS, services HTTP/HTTPS, action ACCEPT
NAT	Desactive dans la policy car le NAT est realise par le VIP

Resolution DNS temporaire

172.20.104.70 prestalab.galaxy-swiss.com
172.20.104.70 webvisite.galaxy-swiss.com

Le poste client doit pointer vers le Fortigate et non directement vers WEBLAB afin de respecter le chemin reseau attendu.

VIII. Déploiement et commandes utiles

Les services sont lancés avec Docker Compose depuis leurs dossiers respectifs. PrestaLab crée le réseau Docker commun, puis Webvisite et Caddy s'y rattachent.

Ordre de démarrage conseillé

1. Démarrer PrestaLab et MariaDB.
2. Démarrer Webvisite.
3. Construire puis démarrer Caddy.
4. Vérifier les conteneurs et le réseau Docker.
5. Tester les URLs depuis un poste client.

Commandes principales

```
cd /srv/web/prestashop  
sudo docker compose up -d
```

```
cd /srv/web/webvisite  
sudo docker compose up -d --build
```

```
cd /srv/web/caddy  
sudo docker compose up -d --build
```

```
sudo docker ps  
sudo docker network inspect prestashop_prestanet  
sudo docker logs caddy --tail 50
```

Correction du domaine PrestaShop si nécessaire

```
docker exec -it prestalab_db mariadb -u root -p  
USE prestalab;  
UPDATE ps_configuration  
SET value='prestalab.galaxy-swiss.com'  
WHERE name IN ('PS_SHOP_DOMAIN','PS_SHOP_DOMAIN_SSL');
```

```
UPDATE ps_shop_url  
SET domain='prestalab.galaxy-swiss.com',  
    domain_ssl='prestalab.galaxy-swiss.com',  
    physical_uri='/'  
WHERE id_shop_url=1;
```

```
docker exec -it prestalab rm -rf /var/www/html/var/cache/*  
docker restart prestalab
```

IX. Tests de validation et securite

Test	Commande / action	Resultat attendu
Conteneurs	<code>docker ps</code>	caddy, prestalab, prestalab_db et webvisite actifs
Reseau Docker	<code>docker network inspect prestashop_prestanet</code>	Tous les conteneurs presents sur le reseau
Caddy -> PrestaLab	<code>docker exec -it caddy curl -I http://prestalab:80</code>	Reponse HTTP du conteneur
Caddy -> Webvisite	<code>docker exec -it caddy curl -I http://webvisite:80</code>	Reponse HTTP ou redirection vers connexion
Acces HTTPS	Navigateur ou <code>curl -k</code>	Page accessible via le nom de domaine

Tests WAF

Test XSS

```
curl -k -I "https://webvisite.galaxy-swiss.com/?test=alert(1)"
```

Test SQLi

```
curl -k -I "https://webvisite.galaxy-swiss.com/?id=1%27%20OR%20%271%27%3D%271"
```

Verification des logs

```
docker logs caddy --tail 50
```

Resultat attendu : le WAF bloque les requetes malveillantes avec un code 403 Forbidden et les logs Caddy indiquent une violation de regle Coraza.

X. Conclusion

La procedure permet de mettre en place une architecture web securisee composee de Docker, Caddy, Coraza WAF, PrestaLab, Webvisite et Fortigate. Les applications sont accessibles en HTTPS par leurs noms de domaine, mais les conteneurs ne sont pas exposes directement au reseau client.

L architecture obtenue respecte les principes de segmentation, centralisation des acces, exposition limitee des services et filtrage applicatif.